

Demos

In this chapter, we will briefly demonstrate the capability of RMF with `rmf_demos`. This will give users a brief understanding of the core features of RMF.

For the most updated `rmf_demos` run instruction, please refer to [here](#).

First make sure, you have have installed the provided RMF demos from their Debian package:

```
# Demos example with gazebo simulator, use ros-foxy-rmf-demos-ign for ignition
sudo apt-get install ros-foxy-rmf-demos-gz
```

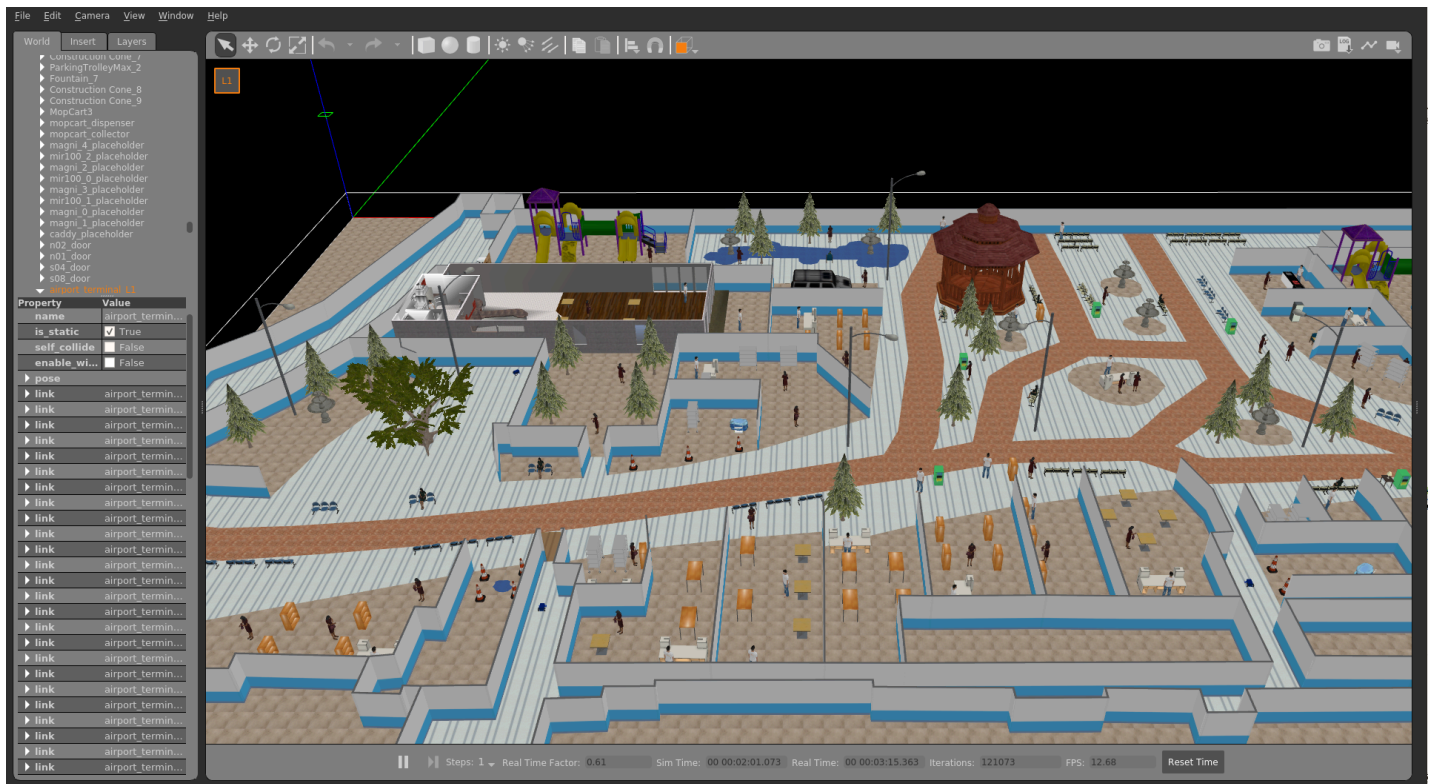
Run your desired demo. In this case we will run the `airport terminal`:

Before running the demo, we can ensure all required models are downloaded locally by:

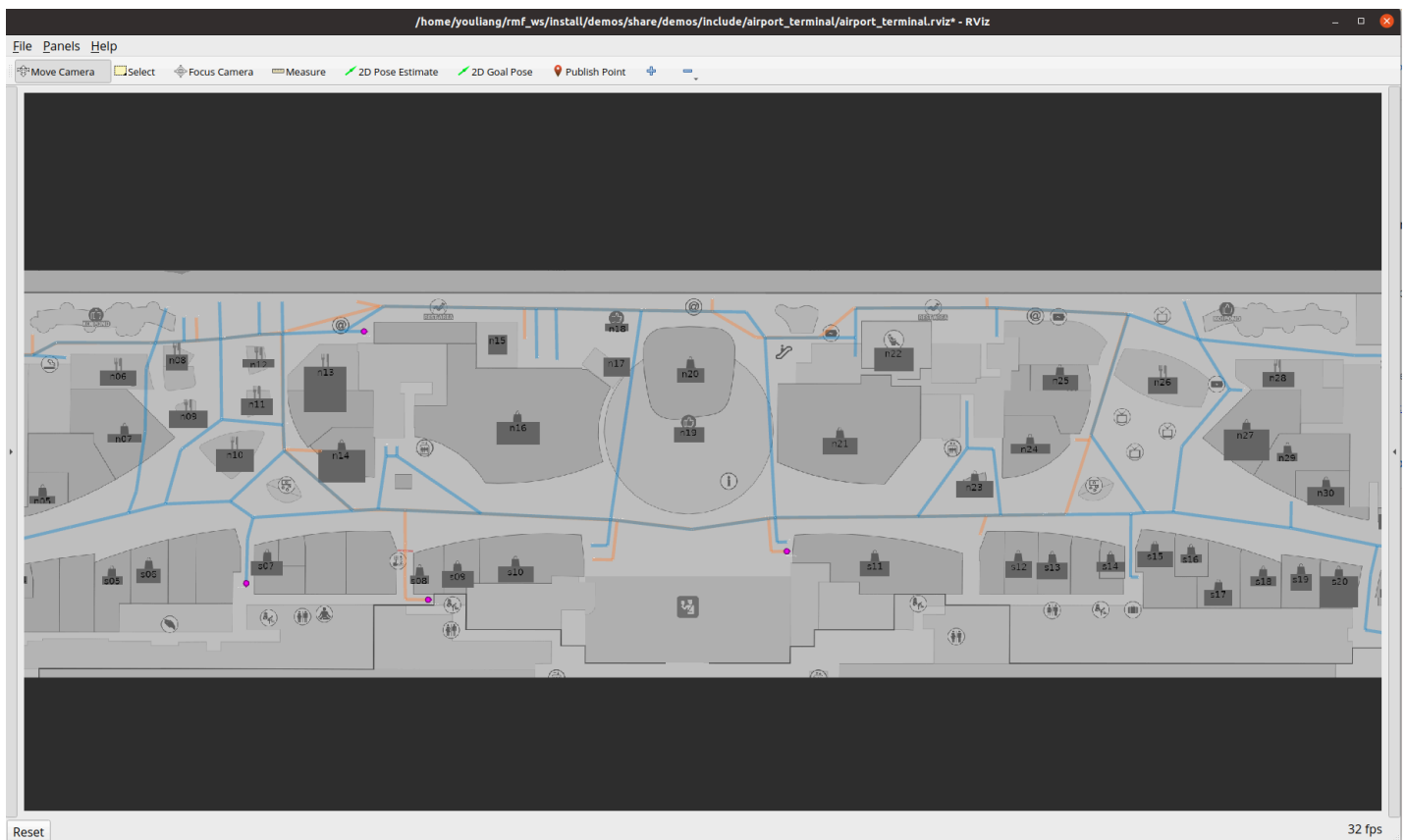
```
ros2 run rmf_building_map_tools model_downloader rmf_demos_maps -s
airport_terminal
```

```
ros2 launch rmf_demos_gz airport_terminal.launch.xml
# or with ignition
ros2 launch rmf_demos_ign airport_terminal.launch.xml
```

Now you should be able to see the airport terminal with the robots in Gazebo:



The RMF Schedule Visualizer should have loaded in an rviz window. This canvas will display all integrated robots or infrastructure which are available to RMF.



During a task request, instead of requiring the user to specify the robot name to complete a task, RMF will assign the task to the best robot.

RMF currently supports 3 types of task, namely: `loop`, `delivery` or `clean`. User can submit a task via CLI:

Loop Task

```
ros2 run rmf_demos_tasks dispatch_loop -s s07 -f n12 -n 3 --use_sim_time
```

Delivery Task

```
ros2 run rmf_demos_tasks dispatch_delivery -p mopcart_pickup -pd mopcart_dispenser
-d spill -di mopcart_collector --use_sim_time
```

Clean Task

```
ros2 run rmf_demos_tasks dispatch_clean -cs zone_3 --use_sim_time
```

Now you can observe robots roaming around the airport space!

rmf_panel Another way to observe and interact with RMF is via a web `rmf_panel`. Open the webpage: `firefox https://open-rmf.github.io/rmf-panel-js/` or click [here](#)

The screenshot shows the RMF Panel web interface in a browser window. The interface has a top navigation bar with tabs for 'Office', 'Airport' (selected), 'Clinic', and 'Hotel'. Below the navigation bar, there's a section for 'Task Submissions' with two main options: (1) 'SUMMIT AD-HOC TASK' and (2) 'SUMMIT TASK LIST'. The 'SUMMIT AD-HOC TASK' section includes a dropdown for 'Select a request type' (set to '0'), a 'Set start time (mins from now)' input (set to '0'), and a 'Choose an evaluator (optional)' dropdown. The 'SUMMIT TASK LIST' section includes a 'Browse...' button, a text area for a task list (containing JSON examples), and a 'SUBMIT TASK LIST' button. Below these sections is a 'Robots & Tasks Summaries' section. It has two tabs: 'Robots' (selected) and 'Tasks'. The 'Robots' tab shows a grid of robot status cards. Each card displays the robot's name, task ID, status, battery level, and location. The robots shown are 'caddy' (Moving-2, 98% battery), 'tinyRobot_1' through 'tinyRobot_4' (all Charging-1, 100% battery), and 'deliveryRobot_0' through 'deliveryRobot_2' (all Charging-1, 100% battery). A 'REFRESH' button is located at the top left of the robots grid.

You can view the status of all the robots under RMF. To request a list of tasks, first select the `Airport` tab. User can choose to submit a (1) Adhoc task or (2) Task List.

Copy paste this to the Task List Box. (or open a file)

```
[{"task_type": "Delivery", "start_time": 0, "description": {"option": "mop"}},
{"task_type": "Loop", "start_time": 0, "description": {"num_loops": 10,
"start_name": "junction_north_west", "finish_name": "n14"}},
{"task_type": "Loop", "start_time": 0, "description": {"num_loops": 10,
"start_name": "tinyRobot_n09", "finish_name": "s07"}},
{"task_type": "Clean", "start_time": 0, "priority": 0, "description":
{"cleaning_zone": "zone_2"}}
]
```

Then Click Submit to submit the list of tasks:

The screenshot shows the RMF Panel web interface in a browser window at localhost:5000. The interface has a top navigation bar with tabs for Office, Airport, Clinic, and Hotel. The main content area is titled 'Task Submissions' and includes a 'Submit a Task' section with a dropdown for 'Select a request type', a 'Set start time (mins from now)' input set to 0, and a 'Choose an evaluator (optional)' dropdown. To the right is a 'Submit a List of Tasks' section with a 'Browse...' button, a text area containing a JSON list of tasks (including Delivery, Loop, and Clean tasks), and a 'SUBMIT TASK LIST' button. Below this is the 'Robots & Tasks Summaries' section. The 'Robots' sub-section shows a grid of robot cards for 'caddy' and various 'tinyRobot' and 'deliveryRobot' units, each displaying task ID, status (e.g., 'Moving-2', 'Charging-1'), battery level, and location. The 'Tasks' sub-section shows a grid of task cards for 'Delivery0', 'Loop1', 'Loop2', and 'Loop3', each displaying task ID, details, robot, task type, task state (e.g., 'Pending'), and start/end times, with a 'CANCEL TASK' button for each.

Now, sit back and enjoy.

Jump in, the water is fine!

So now you have an idea of what RMF is all about, it's time to jump in. We would suggest if you have not already that you take the time to review the [RMF Demos](#) repository and if you want a really quick overview of RMF then take a look at this [Mock Airport Terminal video demo](#) (Short film Oscar nominations most welcome). We hope you find RMF to be a useful tool to help you scale your robot deployments and operations and we look forward to the many improvements and contributions to come!

Found a bug? [Edit this page on GitHub](#).

An Introduction to ROS 2

In this chapter we will cover the basics of Robot Operating System (ROS) and give you all the tools you need to build, debug, and understand robotic applications. This chapter is laid out from the most general concepts, necessary for decision makers to make sound decisions, to specific API references needed by engineers to develop new robotic applications. Somewhere in between high level concepts, and low level API commands lives the knowledge necessary for those maintaining and supporting multi-robot deployments in the field.

A good analogy to learning about ROS is the process of learning about motor vehicles. At the practical, day-to-day level, most people will learn how to start a vehicle and safely use it on a motorway. For these individuals, learning about the high level concepts behind ROS, along with application-specific commands is probably sufficient. Those who enjoy driving often choose to learn how to repair and maintain their vehicle. If this is your interest level we recommend learning the basics of the ROS command line interface. This will allow you to "check the oil" of your robotic systems and make sure everything is functioning correctly. Finally, if you are the type that would like to swap out the engine of your vehicle with something more powerful, or potentially build a wholly new vehicle from scratch, then the ROS API is the set of tools that will make this possible. Generally speaking, automotive engineers don't appear into the world fully formed, and the same is true for roboticists. It is advisable to work through each phase of understanding as you develop your skills with ROS.

Following from our analogy above the process of learning how to use robotic systems built on ROS can be divided roughly into four parts. This chapter works through these four parts of the process, using ROS 2. Subsequent chapters then build upon this knowledge and discuss the subtleties of specific applications. The four parts of this chapter are as follows.

- Meta-discussion of the tools and resources available to help you in the learning process.
- A high level discussion to the design patterns used in ROS. These patterns are roughly analogous to the subsystems you would find in a vehicle (engine, brakes, safety, climate control, etc).
- A treatment of the command line interface (CLI) for ROS. The CLI is a set of programs for starting, inspecting, controlling, and monitoring a ROS robot. You can think of this topic as teaching you how check a robot's oil, and read the instrument panel.
- An introduction to the ROS application programming interface. This section will show you how to create your own applications and modify existing software for your specific application.

While this book aims to cover the basics it should be made clear that ROS, like almost all software, is a moving target. Technology moves quickly, and while print media is helpful at delivering high fidelity instruction, that instruction can become rapidly outdated. For this reason we start this chapter with a meta-discussion of ROS resources that can be used to help you in your learning process.

Found a bug? [Edit this page on GitHub](#).

ROS Kick Off

This chapter describes the avenues available for learning about and getting help with ROS, as well as how to get started by setting up and installing ROS.

Resources

The most up to date information about ROS can be found on the web. There are a myriad of resources on-line to help you out in your educational or practical journey. One thing to keep in mind is that ROS, like most software, has different versions, and the format and structure of commands and API calls may differ slightly between versions (although the developers try to keep things as stable as possible). This book is specifically written for *ROS 2, Eloquent Elusor*, or ROS Eloquent to be terse.

While newer or older versions of ROS 2 will be generally similar, it is worth paying attention to the distribution name or version number as there are changes between versions. A major version of ROS 2 generally corresponds to a distribution, denoted by a pair of matching letter adjectives and specific nouns related to specific genus and species of turtles (e.g. Eloquent Elusor, or Foxy Fitzroy). It is worth noting that ROS versions are usually pegged to specific version of Ubuntu Linux.

ROS grew up with the modern web, and as such it has a variety of resources and forums to help you solve problems and learn about the API and tools. Some of our web resources actually pre-date more widely used systems, so it helps to know where they are and how to use them. Probably the most important resource on the web for ROS users is answers.ros.org. Answers is a Q&A website similar to StackOverflow. Once you register for Answers you can ask or answer any ROS-related question. Be aware that asking a question well can be difficult. You should include as much information as possible to help others answer your question. This means you should include the ROS version, platform version, any debugging or stack trace information you have, and the offending source code.

Aside from ROS Answers you should check out both the ROS 2 tutorials and API documentation, and the ROS 1 wiki. The ROS 1 wiki can be found at wiki.ros.org. While it is specifically dedicated to ROS 1, much of the information is still relevant to ROS 2. If you are searching for up to date ROS 2 information, your go to source is the ROS 2 tutorials and API documents located at index.ros.org/doc/ros2. Many of the tutorials you will find in this book pull directly from this body of work. If you would like to find the latest ROS news and discuss various ROS features, the ROS Discourse forum at discourse.ros.org is your best bet. ROS

discourse is the community hub where developers discuss their latest projects and debate the finer points of ROS development.

For ROS application developers there are a number of tools to help you connect with the broader ROS developer community. Open Robotics supports index.ros.org, which is an extended list of ROS packages sorted by version. If you are searching for a ROS driver for a particular piece of hardware, then the index is a great place to start. If you find a package with failing tests, or would like to know the build status of any ROS package, take a look at build.ros.org. Similarly, for un-indexed packages [GitHub maintains a ROS code tag](#). This tag will allow you to search all of the tagged repositories that are publicly listed. At the time of writing there were close to 4000 repositories listed on GitHub, so there is a pretty good chance you'll find what you need.

Finally, there are a variety of unofficial resources that you should be aware of that can be useful, particularly if you want to keep yourself up to date with the latest ROS projects and features. Both [Open Robotics](#) and [ROS](#) maintain twitter feeds to share the latest news. We also have a yearly ROS developers conference called [ROSCon](#); most talks are freely available on the web. There are a few other resources that can also be useful including the [ROS subreddit](#) and an "unofficial" [ROS Discord](#).

Setting Up Your Computer

For this chapter we assume that you are working on a modern desktop with a discrete graphics card. While a graphics card isn't necessary for this chapter, later chapters will be graphics intensive and having one will greatly improve the end user experience. Moreover, this book assumes you are working with the Ubuntu Linux 18.04 operating system. While other operating systems are supported by ROS 2, all of the tutorials and instructions on this book assume you are running Linux. If instead you're using a Mac or Windows PC, you can install ROS 2 Eloquent Elusor using the instructions found on the [ROS 2 installation instructions](#) page. An alternative path for installation on Mac and PC is to using a virtual machine. Roughly the process for doing so is as follows:

1. Install virtual machine software like [Virtual Box](#) or [VMWare](#) on your host machine.
2. Create a virtual machine using the software, and install [Desktop Ubuntu 18.04 Bionic Beaver from the Canonical website](#). Configure the installation as you wish.
3. Now start your virtual machine and log in as a user. The directions below should be applicable.

For these initial tutorials we will be working with the pre-compiled ROS 2: Eloquent Elusor desktop version. These directions follow directly from the installation instructions found on the [\[Eloquent installation page\]](https://index.ros.org/doc/ros2/Installation/Eloquent/Linux-Install-)<https://index.ros.org/doc/ros2/Installation/Eloquent/Linux-Install->

Debian(s)). To run these commands you'll need a terminal window. To open a terminal in Ubuntu 18.04 click on the nine dots in the bottom left hand of the screen. A dialog should appear. Enter the word *terminal* and click on the terminal icon to open a terminal. Alternatively, you can press the control, alt, and 't' keys simultaneously to open a terminal (we abbreviate this CTRL-ALT-T).

Setup Locale

The first step is to make sure you have a locale which supports UTF-8 . What this means is that we will check that the language used by your computer uses a particular format of text. If you are in a minimal environment, such as a Docker container, the locale may be something minimal like POSIX. We test with the following settings. It should be fine if you're using a different UTF-8 supported locale.

```
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

Setup Sources

You will need to add the ROS 2 apt repositories to your system. Out of the box Ubuntu doesn't know where the ROS 2 binary programs live so we have to give it a secure location. To do this the computer will prompt you for your root password. For more technical readers we need to authorize the ROS GPG key with apt by typing the following command in the terminal:

```
sudo apt update && sudo apt install curl gnupg2 lsb-release
curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo apt-
key add -
```

Install ROS 2 Packages

The next steps for installing ROS is to do a system update (i.e. check for newer programs) and then install ROS Eloquent. To do this we run the following commands. Be aware that these commands will download a lot of data and may take awhile. It is best to run these commands on your home network.

```
sudo apt update
```

Desktop Install (Recommended): ROS, RViz, demos, tutorials.

```
sudo apt install ros-eloquent-desktop
```

Next we'll install a set of tutorials called `TurtleSim`. To do this we run another apt command.

```
sudo apt install ros-eloquent-turtlesim
```

ROS 2 command line tools use `argcomplete` for autocompletion. If you want autocompletion, installing `argcomplete` is necessary. We're also going to install a few other tools to make our lives easier.

```
sudo apt install python3-argcomplete htop byobu
```

Check Your Installation

ROS uses `environment variables` to help keep track of what version of ROS is running and where all the programs using ROS are on the computer. To set these environment variables we `source`, or load, a bash script file. A bash script file isn't magic; it is a series of commands to enter into the terminal, just like the series of commands we just entered to setup ROS. It is possible to have different versions of ROS running on a single computer. Using the wrong version of ROS can lead to all sorts of problems and is a common mistake for new users! If you are having problems, try sourcing the correct ROS bash file. From now on, whenever you open a new terminal, you will need to tell the computer which version of ROS to use. To set the necessary environment variables for ROS you need to `source` a bash file every time you open a new terminal. Yes, this is annoying, but it is a sound approach as it makes the version of ROS you are using explicit. On Ubuntu 18.04 all versions of ROS live in `/opt/ros/`. Inside this directory will be a programs and scripts file to run ROS. To tell the operating system that we want to use ROS Eloquent we simply source the ROS Eloquent `setup.bash` file using the command below:

```
source /opt/ros/eloquent/setup.bash
```

Once that command runs, your terminal should be ready to run a ROS program. Let's test our installation by running two small ROS programs called `talker` and `listener`. These two programs will send data back and forth using ROS to perform the communication. One program was written in C++ and the other in Python. Running these two different programs is a